

Lista comentada

Análise, projeto e complexidade de algoritmos

Estudo de caso: busca de matrículas em uma secretaria acadêmica

Objetivo da atividade

Esta atividade tem como objetivo ajudar o estudante a compreender, de modo elementar, como diferentes algoritmos podem resolver o mesmo problema com desempenhos diferentes.

- Identificar a entrada e a saída de um problema computacional.
- Projetar soluções usando pseudocódigo.
- Comparar busca linear, busca binária e estrutura hash.
- Analisar a complexidade usando notação O .
- Escolher o algoritmo mais adequado de acordo com o cenário.

Situação-problema

Uma secretaria acadêmica possui uma lista de matrículas de estudantes e precisa consultar se uma matrícula específica está cadastrada.

```
Lista de matrículas:  
[2023001, 2023045, 2023102, 2023120, 2023201, 2023333]  
  
Pergunta:  
A matrícula 2023102 está cadastrada?
```

Parte A: Análise inicial

Questão 1: Qual é a entrada do problema?

Resposta esperada:

A entrada do problema é uma lista de matrículas e uma matrícula que será procurada.

Comentário:

O algoritmo precisa saber onde procurar e qual valor procurar. Sem esses dois dados, o problema não pode ser resolvido.

Questão 2: Qual é a saída esperada?

Resposta esperada:

A saída esperada é verdadeiro, se a matrícula estiver na lista, ou falso, se a matrícula não estiver na lista.

Comentário:

A resposta não precisa devolver a posição da matrícula. A pergunta é apenas se a matrícula existe ou não.

Questão 3: O problema exige apenas uma consulta ou várias consultas?

Resposta esperada:

O problema pode envolver uma única consulta ou várias consultas.

Comentário:

Essa informação é importante porque uma solução simples pode ser suficiente para poucas consultas, mas pode ser ruim quando há muitas consultas.

Questão 4: O tamanho da lista influencia o desempenho?

Resposta esperada:

Sim. Quanto maior a lista, maior pode ser o tempo necessário para encontrar uma matrícula.

Comentário:

Em uma lista pequena, procurar item por item é rápido. Em uma lista grande, essa estratégia pode ficar lenta.

Parte B: Projeto dos algoritmos

Solução 1: Busca linear

```
para cada matrícula na lista:
    se matrícula for igual à matrícula procurada:
        retornar verdadeiro

retornar falso
```

Comentário:

A busca linear percorre a lista do começo ao fim. No pior caso, ela precisa verificar todas as matrículas.

Complexidade: $O(n)$.

Solução 2: Busca binária

```
ordenar a lista

início = 0
fim = tamanho da lista - 1

enquanto início <= fim:
    meio = (início + fim) / 2

    se lista[meio] for igual à matrícula procurada:
        retornar verdadeiro

    se matrícula procurada for menor que lista[meio]:
        fim = meio - 1
    senão:
        início = meio + 1

retornar falso
```

Comentário:

A busca binária só funciona corretamente quando a lista está ordenada. Ela divide o espaço de busca ao meio a cada passo.

Complexidade da ordenação: $O(n \log n)$.

Complexidade da busca: $O(\log n)$.

Solução 3: Estrutura hash

```
criar um conjunto vazio

para cada matrícula na lista:
    inserir matrícula no conjunto

verificar se a matrícula procurada pertence ao conjunto
```

Comentário:

A estrutura hash é adequada quando queremos fazer muitas consultas. Em Python, usamos set. Em Java, usamos HashSet.

Complexidade para criar a estrutura: $O(n)$.

Complexidade média de cada consulta: $O(1)$.

Parte C: Implementação comentada

Implementação em Python

```
def busca_linear(lista, alvo):
    for item in lista:
        if item == alvo:
            return True
    return False

def busca_binaria(lista, alvo):
    lista = sorted(lista)
    inicio = 0
    fim = len(lista) - 1

    while inicio <= fim:
        meio = (inicio + fim) // 2

        if lista[meio] == alvo:
            return True
        elif alvo < lista[meio]:
            fim = meio - 1
        else:
            inicio = meio + 1

    return False

def busca_hash(lista, alvo):
    conjunto = set(lista)
    return alvo in conjunto
```

Comentário:

A função `busca_linear` percorre a lista item por item. A função `busca_binaria` ordena a lista e depois faz divisões sucessivas. A função `busca_hash` transforma a lista em um conjunto e consulta diretamente se o valor existe.

Implementação em Java

```
import java.util.*;

public class BuscaMatricula {

    public static boolean buscaLinear(int[] lista, int alvo) {
        for (int item : lista) {
            if (item == alvo) {
                return true;
            }
        }
    }
}
```

```

    }
    return false;
}

public static boolean buscaBinaria(int[] lista, int alvo) {
    Arrays.sort(lista);

    int inicio = 0;

    int fim = lista.length - 1;

    while (inicio <= fim) {
        int meio = (inicio + fim) / 2;

        if (lista[meio] == alvo) {
            return true;
        } else if (alvo < lista[meio]) {
            fim = meio - 1;
        } else {
            inicio = meio + 1;
        }
    }

    return false;
}

public static boolean buscaHash(int[] lista, int alvo) {
    HashSet<Integer> conjunto = new HashSet<>();

    for (int item : lista) {
        conjunto.add(item);
    }

    return conjunto.contains(alvo);
}
}

```

Comentário:

Em Java, `Arrays.sort` ordena o vetor. `HashSet` permite guardar os valores em uma estrutura própria para consultas rápidas.

Parte D: Resultado da análise de complexidade

Estratégia	Preparação	Consulta	Resultado em uma consulta	Quando usar
------------	------------	----------	---------------------------	-------------

Busca linear	$O(1)$	$O(n)$	$O(n)$	Poucos dados ou poucas consultas
Busca binária	$O(n \log n)$	$O(\log n)$	$O(n \log n + \log n)$	Lista já ordenada ou muitas consultas
HashSet/set	$O(n)$	$O(1)$ em média	$O(n)$ para preparar e consultar uma vez	Muitas consultas rápidas

Comentário:

A busca linear não precisa preparar os dados, mas cada consulta pode ser lenta. A busca binária exige ordenação, mas depois consulta rapidamente. A estrutura hash exige criação inicial, mas depois responde consultas em tempo médio constante.

Parte E: Resultado por cenário

Cenário	Melhor escolha esperada	Justificativa
Uma consulta por dia	Busca linear	Como há apenas uma consulta, não compensa ordenar a lista nem criar uma estrutura hash.
500 consultas por dia	Busca binária ou HashSet/set	Como há várias consultas, começa a compensar preparar os dados antes.
50.000 consultas por dia	HashSet em Java ou set em Python	Como há muitas consultas, a criação da estrutura hash tende a compensar pelo ganho nas buscas.

Conclusão esperada

O melhor algoritmo depende da situação. Para poucas consultas, a busca linear pode ser suficiente. Para muitas consultas, vale a pena preparar os dados, usando ordenação ou uma estrutura hash.

De modo geral, para muitas buscas, HashSet em Java ou set em Python tende a ser a solução mais eficiente.

Rubrica de correção

Critério	Pontuação	O que observar
----------	-----------	----------------

Compreensão do problema	1,0	Identifica corretamente entrada, saída e objetivo da busca.
Pseudocódigo	1,5	Apresenta lógica coerente para as três soluções.
Implementação	2,0	O código executa corretamente e retorna verdadeiro ou falso.
Complexidade	2,0	Usa corretamente $O(n)$, $O(\log n)$, $O(n \log n)$ e $O(1)$.
Comparação	2,0	Justifica qual algoritmo usar em cada cenário.
Clareza	1,5	Explica com organização e linguagem compreensível.
Total	10,0	

Síntese para o estudante

- Busca linear: simples, mas pode ser lenta em listas grandes.
- Busca binária: rápida, mas precisa de lista ordenada.
- HashSet/set: muito rápido para consultas, mas precisa criar a estrutura antes.
- A escolha do algoritmo depende do tamanho da lista e da quantidade de consultas.